- The number of bits used to store a key doesn't depend on the key's size, not on the number of elements already added to the container.

Given these assumptions, the running time for `insert(X)` is `O(k*T(|X|))`: for each of the `k` hash functions we need, in fact, to generate a hash value from the key and store a single bit; if keys are numbers, for instance integers or doubles, then `|X|=1` and `T(|X|) = O(1)`: we can typically generate a hash value in constant time.

If, however, our keys are of variable length - for example strings - then computing each hash value requires linear time in the length of the string: so, in this case, `T(|X|) = O(|X|)`, and the running time will depend on the length of the keys we add.

Now, suppose we know that the longest key will have at most `z` characters, where `z` is a constant. Remembering our assumption on the length of keys being independent on anything else, we can then still argue that running time for `insert(X)` is `O(k*(1+z)) = O(k)`. Thus, this is a constant time operation, no matter how many elements have been already added to the container.

4.9.4 Looking Up an Element

The same considerations hold true for the lookup of a key: we need to transform keys into a set of indices (done in time `O(z*k)`), and then check each bit at those indices (`O(k)` time for all of them). So, lookup as well is a constant time